Tugas Implementasi Transformer

Nama : Putri Torsia Aura Prameswary
NIM : 2546000083
Mata Kuliah / Kelas : Pembelajaran Mesin / B

1. **Dataset yang digunakan**
   - 1.1. **Text Classification:** Menggunakan dataset IMDB Movie Reviews yang berisi 50.000 ulasan film berbahasa Inggris dengan 2 kelas sentimen: positif dan negatif. Dataset diambil dari file lokal /content/IMDB Dataset.csv dan berhasil dimuat seluruhnya. Teks dibersihkan dengan cara dikonversi ke huruf kecil, dihapus tag HTML, dan dibuang karakter selain huruf. Vocabulary dibangun dari 15.000 kata paling sering muncul, dan setiap kalimat di-tokenisasi lalu dipadding hingga panjang maksimum 120 token. Pada tahap pelatihan model digunakan 10.000 sampel dengan split 80:20 untuk training dan testing. Link dataset: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
   - 1.2. **Vision Classification:** Menggunakan dataset MNIST yang terdiri dari 70.000 gambar tulisan tangan angka 0 sampai 9 dalam format grayscale. Dataset diunduh otomatis melalui torchvision.datasets.MNIST, kemudian di-resize ke 32x32 piksel dan dinormalisasi dengan mean dan std 0.5 agar kompatibel dengan arsitektur Vision Transformer yang digunakan. Link dataset: http://yann.lecun.com/exdb/mnist/
   - 1.3. **Non-Klasifikasi (GitHub):** Menggunakan repository Real-ESRGAN dari xinntao (https://github.com/xinntao/Real-ESRGAN) untuk tugas Image Super-Resolution, yaitu meningkatkan resolusi dan kualitas gambar yang buram atau terdegradasi menjadi gambar tajam beresolusi tinggi. Input yang digunakan adalah gambar karakter anime kucing robot berwarna yang tampak buram dan berbintik.
2. **Model yang diimplementasikan**
   - 2.1. **Text Transformer:** Mengimplementasikan arsitektur Transformer dari nol dengan empat komponen utama: Embedding Layer yang mengubah token integer menjadi vektor 128 dimensi, Positional Encoding berbasis fungsi sinus dan kosinus untuk menyuntikkan informasi posisi kata, TransformerEncoder dengan 2 layer dan 8 attention head yang memproses seluruh token secara paralel, serta Linear Classifier yang menerima hasil mean pooling untuk menghasilkan prediksi akhir. Hyperparameter yang dipakai: `embed_dim=128`, `nhead=8`, `num_layers=2`, `dropout=0.3`, `learning rate=0.0005`, `batch_size=64`, `epochs=5`.
   - 2.2. **Text Baseline:** Mengimplementasikan LSTM 2 layer sebagai model pembanding pada domain teks, dengan `embed_dim=128`, `hidden_dim=128`, dan `dropout=0.2`. Prediksi diambil dari hidden state terakhir layer paling atas. Learning rate yang digunakan adalah `0.001`.

2.3. Vision Transformer (ViT): Mengimplementasikan ViT dari nol dengan alur: gambar dipotong menjadi patch 4x4 menggunakan Conv2d sehingga menghasilkan 64 patch per gambar, ditambahkan CLS token di depan urutan patch sebagai representasi global gambar, kemudian Positional Embedding yang bisa dipelajari ditambahkan ke seluruh urutan. TransformerEncoder dengan 3 layer dan 8 attention head memproses seluruh patch, dan MLP Head menghasilkan prediksi untuk 10 kelas digit. Hyperparameter: `patch_size=4, embed_dim=64, nhead=8, num_layers=3, learning rate=0.001, epochs=5`.

2.4. Vision Baseline: Mengimplementasikan SimpleCNN sebagai model pembanding pada domain gambar. Arsitekturnya terdiri dari dua blok konvolusi (Conv2d 1 ke 16 channel dan Conv2d 16 ke 32 channel, diikuti ReLU dan MaxPool2d) lalu dua fully-connected layer dari 32x8x8 ke 128 ke 10 kelas.

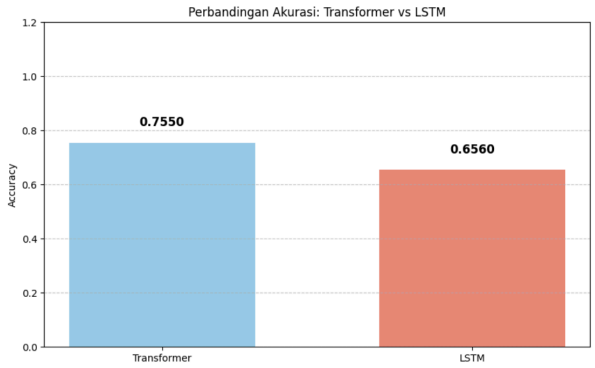
2.5. Non-Klasifikasi: Mengimplementasikan Real-ESRGAN menggunakan bobot pretrained RealESRGAN_x4plus.pth (63,93 MB) yang diunduh dari GitHub Releases. Model menggunakan generator berbasis Residual-in-Residual Dense Block (RRDB) yang dilatih dengan GAN untuk merekonstruksi detail halus dari gambar terdegradasi. Inferensi dijalankan menggunakan skrip resmi inference_realesrgan.py dengan dua variasi parameter skala: 4x dan 2x.

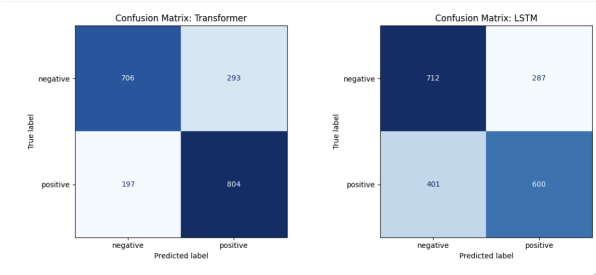
3. Hasil eksperimen

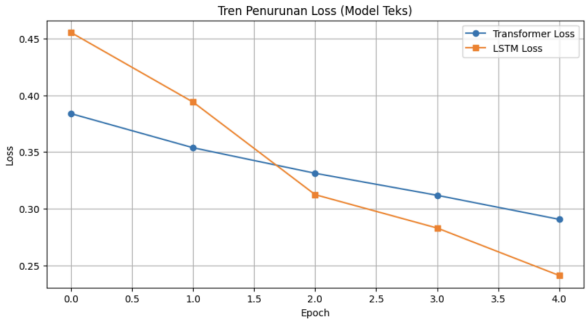
3.1. Tabel 1. Perbandingan Model Text dan Vision

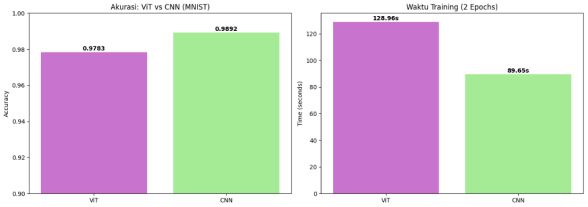
Berikut adalah rekapitulasi hasil metrik setelah pelatihan model dengan 5 epoch.

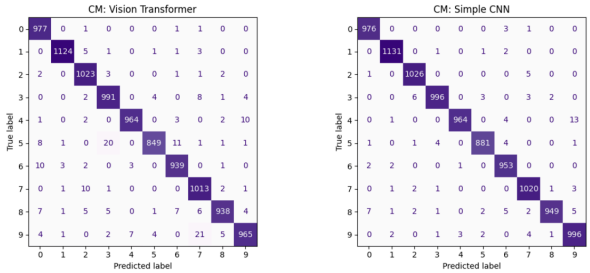
Model	**Dataset**	**Accuracy (Test)**	**Loss (Epoch 5)**	**Parameter**	**Waktu Training**	**Catatan**
Transformer Text	IMDB (10.000 sampel)	75,50%	~0,292	2.466.306	Lebih lambat	Underfitting ringan, perlu data lebih banyak.
LSTM (Text Baseline)	IMDB (10.000 sampel)	65,60%	~0,242	1.412.354	Lebih cepat	Banyak false negative pada kelas positif.
Vision Transformer (ViT)	MNIST	97,83%	Turun stabil	849.418	128,96 detik	Menggunakan patch size 4x4.
Simple CNN	MNIST	98,92%	Turun cepat	268.362	89,65 detik	Sangat efisien dan akurat berkat inductive bias.

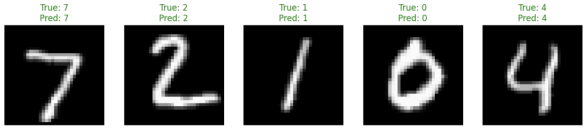










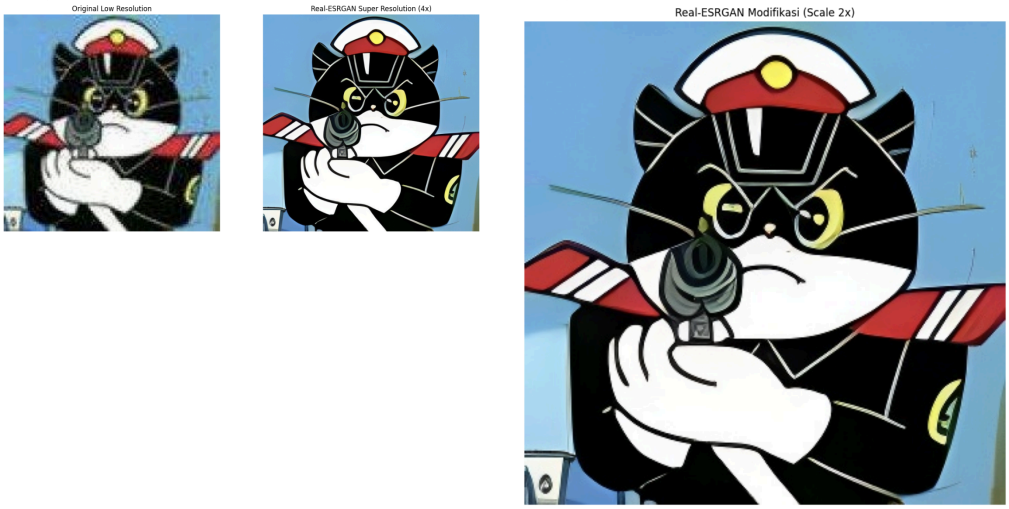


4. Implementasi GitHub non-klasifikasi

4.1. Tabel 2. Detail Implementasi GitHub

Aspek	Isi
Link Repository	https://github.com/xinntao/Real-ESRGAN
Jenis Tugas	Image Super-Resolution (Restoration)
Model	Real-ESRGAN dengan bobot RealESRGAN_x4plus.pth (63,93 MB)
Dataset/Input	Gambar karakter anime kucing robot beresolusi rendah
Output Model	Gambar beresolusi tinggi hasil peningkatan skala 4x dan 2x

Modifikasi Kode	Mengubah parameter `--outscale` dari 4 menjadi 2; menambahkan patch kompatibilitas pada basicsr untuk torchvision versi baru
Hasil Setelah Modifikasi	Scale 4x: gambar sangat tajam dan detail besar. Scale 2x: lebih cepat diproses, ukuran lebih kecil, namun kualitas tetap jauh melebihi gambar aslinya.



5. Modifikasi kode

- Pada eksperimen teks, Transformer awalnya mengalami mode collapse. Tiga perubahan diterapkan: jumlah data training dinaikkan ke 10.000 sampel, learning rate diperkecil dari 0.001 menjadi 0.0005, dan fungsi train_with_loss ditambahkan untuk memantau loss per epoch lewat grafik.
- Pada Real-ESRGAN, dua modifikasi dilakukan. Pertama, patch kompatibilitas pada file basicsr/data/degradations.py untuk mengganti impor yang deprecated agar program bisa berjalan di Python 3.12. Kedua, parameter --outscale diubah dari 4 menjadi 2.
    - Sebelum modifikasi: --outscale 4, impor torchvision.transforms.functional_tensor.
    - Sesudah modifikasi: --outscale 2, impor diganti ke torchvision.transforms.functional.

  Hasilnya: scale 4x menghasilkan gambar sangat tajam dan besar, scale 2x lebih ringan dan cepat namun tetap jauh lebih baik dari aslinya.

6. Analisis hasil

6.1. Model mana yang paling akurat?

Pada domain teks, Transformer lebih unggul dari LSTM pada eksperimen 10.000 sampel (75,50% vs 65,60%), namun pada 50.000 sampel posisinya berbalik karena LSTM mencapai 85,33% dan Transformer 80,44%. Ini menunjukkan LSTM lebih stabil di berbagai ukuran data. Pada domain vision, CNN lebih akurat

dari ViT (98,92% vs 97,83%) meski jumlah parameternya tiga kali lebih sedikit (268.362 vs 849.418).

6.2. Model mana yang paling cepat?

CNN adalah model tercepat dengan waktu training 89,65 detik untuk 5 epoch, dibanding ViT yang membutuhkan 128,96 detik. Selisih sekitar 39 detik ini mencerminkan overhead dari mekanisme self-attention ViT yang memiliki kompleksitas kuadratik terhadap jumlah patch. Pada domain teks, LSTM konvergen lebih cepat secara grafik loss karena tidak perlu menghitung relasi seluruh pasangan token sekaligus.

6.3. Apakah Transformer selalu lebih baik?

Tidak. Eksperimen ini membuktikannya secara langsung. Pada 10.000 sampel IMDB, LSTM punya loss akhir lebih rendah (0,242 vs 0,292). Di domain vision, CNN lebih akurat dan lebih cepat dari ViT dengan parameter tiga kali lebih sedikit. Transformer baru menunjukkan potensi penuhnya pada skala data sangat besar atau saat digunakan sebagai fondasi transfer learning seperti BERT atau ViT-L/16.

6.4. Mengapa CNN masih kuat untuk gambar kecil?

CNN punya inductive bias berupa translation invariance dan locality yang secara alami cocok untuk gambar resolusi rendah seperti MNIST (32x32 piksel). Dengan hanya 268.362 parameter, CNN mencapai akurasi 98,92%, lebih tinggi dari ViT yang memiliki tiga kali lebih banyak parameter. Prior knowledge yang tertanam dalam arsitektur CNN jauh lebih menguntungkan dibanding fleksibilitas generik ViT untuk gambar kecil.

6.5. Mengapa LSTM/CNN 1D masih relevan untuk teks pendek?

Untuk teks dengan panjang maksimal 120 token, LSTM mampu menangkap dependensi sekuensial secara efisien. Dengan 1.412.354 parameter, LSTM tetap kompetitif dibanding Transformer yang punya hampir dua kali lebih banyak parameter. Jumlah parameter yang lebih besar tidak otomatis menghasilkan akurasi yang lebih tinggi, terutama saat data pelatihan terbatas.

6.6. Apa kelemahan Transformer pada dataset kecil?

Transformer sangat rentan terhadap mode collapse saat data pelatihan sedikit. Dalam eksperimen ini masalah tersebut benar-benar terjadi dan membutuhkan penanganan eksplisit berupa penambahan data dan penurunan learning rate. Performa Transformer juga sangat bergantung pada jumlah data: pada 50.000 sampel akurasinya 80,44%, namun turun ke 75,50% pada 10.000 sampel saja.

6.7. Apa pengaruh patch size pada ViT?

Patch size 4x4 menghasilkan 64 patch per gambar, sehingga ViT memproses sequence sepanjang 65 token (64 patch ditambah 1 CLS token). Semakin kecil patch, semakin banyak detail yang ditangkap namun beban komputasi naik secara kuadratik. Ini tercermin dari waktu training ViT yang 44% lebih lama dari CNN meski resolusi gambar MNIST tergolong rendah.

6.8. Apa pengaruh sequence length pada teks?

Dengan panjang 120 token, attention matrix yang dihitung berukuran 120x120. Dari grafik loss terlihat Transformer memulai epoch pertama dengan loss lebih

rendah (0,384) dibanding LSTM (0,457), menandakan Transformer lebih cepat menangkap konteks panjang di awal. Namun konvergensinya lebih lambat dan loss akhirnya masih di atas LSTM.

6.9. Apa input dan output repository GitHub non-klasifikasi?

Input berupa gambar karakter anime kucing robot yang tampak buram, berbintik, dan beresolusi rendah. Output berupa gambar yang sama dalam resolusi jauh lebih tinggi: scale 4x menghasilkan gambar sangat tajam dengan tepi bersih, sementara scale 2x menghasilkan gambar berukuran lebih kecil namun tetap jauh lebih tajam dari input aslinya.

6.10. Mengapa tugas GitHub tersebut bukan klasifikasi?

Karena outputnya bukan label atau kategori, melainkan gambar piksel baru hasil rekonstruksi. Model tidak memutuskan gambar ini termasuk kelas apa, melainkan bagaimana setiap piksel seharusnya terlihat dalam resolusi tinggi. Ini adalah tugas generative dan restoration, bukan klasifikasi.

6.11. Modifikasi apa yang dilakukan dan bagaimana pengaruhnya?

Dua modifikasi dilakukan pada Real-ESRGAN. Patch kompatibilitas pada basicsr memungkinkan program berjalan di Python 3.12 tanpa error pada impor yang deprecated. Perubahan `--outscale` dari 4 ke 2 menghasilkan output yang bisa dibandingkan langsung: scale 4x menghasilkan gambar lebih besar dan sangat detail, scale 2x lebih ringan dan cepat namun kualitasnya tetap jauh melebihi input asli. Perbandingan ini memperlihatkan secara konkret bagaimana satu parameter memengaruhi trade-off antara kualitas, ukuran, dan kecepatan komputasi.

7. Kesimpulan

Eksperimen ini membuktikan bahwa Transformer bukan selalu pilihan terbaik. Pada dataset teks berukuran menengah, LSTM tetap kompetitif dan lebih stabil. Pada gambar resolusi rendah, CNN lebih akurat dan lebih efisien dibanding ViT dengan parameter tiga kali lebih sedikit. Transformer baru menunjukkan keunggulan nyata pada skala data besar atau sebagai fondasi transfer learning.

Implementasi Real-ESRGAN membuktikan penerapan nyata model deep learning untuk tugas non-klasifikasi, di mana perubahan satu parameter skala langsung mengubah karakteristik output secara terukur. Tujuan utama tugas ini bukan mencari angka akurasi tertinggi, melainkan memahami cara kerja setiap arsitektur dan melatih kemampuan membaca serta memodifikasi kode dari proyek nyata.

8. Referensi / sumber kode

1. Repository GitHub Pengumpulan: https://github.com/putriprameswary/implementasi-transformer-text-vision
2. Repository Real-ESRGAN: https://github.com/xinntao/Real-ESRGAN
3. Dataset IMDB: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
4. Dataset MNIST: http://yann.lecun.com/exdb/mnist/
5. PyTorch Documentation: https://pytorch.org/docs/stable/index.html
6. Scikit-learn Documentation: https://scikit-learn.org/stable/